

PE 정적 특징 추출기 작성과 생성형AI 분석 결과 검증

1. 개요

강사가 배포한 PE 샘플 6개(정상 3개 / 악성 의심 3개)를 대상으로 정적 특징 추출 스크립트 `extract_features.py` 를 작성하고, 그 결과를 `features.csv` 로 저장했다. 이어서 생성형AI(Claude, ChatGPT)에게 각 샘플의 악성 여부 판정과 근거를 요청한 뒤, AI가 근거로 제시한 API 이름, 섹션 이름, 수치가 실제 추출 결과와 일치하는지 항목별로 대조했다.

분석 대상 6개 샘플은 저장소의 `PE_6/` 디렉터리에 있으며, 무결성 확인용 SHA-256 해시는 `PE_6/SHA256SUMS.txt` 에 기록했다.

모든 분석은 정적 분석이며 어떤 샘플도 실행하지 않았다. `extract_features.py` 는 파일을 바이너리 모드로 열어 (`open(path, "rb")`) 바이트를 읽거나 `pefile.PE()` 로 헤더 구조체를 파싱할 뿐이고, `subprocess`, `os.system`, `exec` 등 샘플을 프로세스로 구동하는 코드는 포함되어 있지 않다.

2. 추출 방법

2.1 요구사항별 구현

요구사항	구현 함수	방법
MZ/PE 시그니처 확인	<code>check_mz_pe_signature()</code>	파일 선두 2바이트가 <code>MZ</code> 인지 확인하고, DOS 헤더 <code>0x3C</code> 오프셋의 <code>e_lfanew</code> 값이 가리키는 위치에 <code>PE\0\0</code> 시그니처가 있는지 원시 바이트로 직접 검사
32/64비트 판별	<code>get_bit_width()</code>	<code>IMAGE_FILE_HEADER.Machine</code> 값이 <code>0x14C</code> (i386)이면 32비트, <code>0x8664</code> (AMD64)이면 64비트
섹션 이름/크기/엔트로피	<code>extract_sections()</code> , <code>calc_shannon_entropy()</code>	섹션 이름은 8바이트 고정 길이에서 NULL 패딩 제거, 크기는 <code>SizeOfRawData</code> , 엔트로피는 라이브러리 없이 직접 구현
Import DLL/API 추출	<code>extract_imports()</code>	<code>pefile</code> 의 <code>DIRECTORY_ENTRY_IMPORT</code> 순회. 해당 속성 자체가 없으면(패킹된 파일에서 흔함) 빈 목록 반환
의심 API 대조	<code>analyze_sample()</code>	추출된 API 집합과 과제 지정 의심 API 10종의 교집합
<code>features.csv</code> 저장	<code>main()</code>	지정된 8개 열 구성, 샘플당 1행

2.2 엔트로피 직접 구현

과제 요구사항에 따라 엔트로피 계산에는 외부 라이브러리를 쓰지 않았다. `calc_shannon_entropy()` 는 섹션의 원시 바이트에서 바이트 값 0부터 255까지 각각의 출현 횟수를 세고, 각 값의 확률 $p = (\text{해당 값의 개수}) / (\text{전체 바이트 수})$ 를 구한 뒤 $-\sum p \cdot \log_2(p)$ 를 합산한다. 값의 범위는 0(모든 바이트가 동일)부터 8(256개 값이 완전 균등 분포)까지다.

구현이 맞는지 다음 두 가지로 검증했다.

- **이론값 대조:** 256개 값이 32회씩 균등하게 등장하는 데이터는 8.0, 단일 값만 반복되는 데이터는 0.0이 나와야 한다. 실제로 `bytes(range(256))*32` 는 정확히 `8.0` , `b'A'*1000` 은 정확히 `0.0` 을 반환했다.
- **독립 구현 교차검증:** 동일한 수식을 `collections.Counter` 기반의 다른 방식으로 다시 구현해 12개 섹션 전체를 계산한 결과, 소수점 12자리까지 모두 일치했다.

2.3 의심 API 목록

과제에 지정된 10종을 그대로 사용했다: `VirtualAlloc` , `VirtualProtect` , `WriteProcessMemory` , `CreateRemoteThread` , `LoadLibraryA` , `GetProcAddress` , `RegSetValueExA` , `InternetOpenA` , `CryptEncrypt` , `ShellExecuteA` .

코드 전체는 `extract_features.py` 에 있으며 각 함수와 단계마다 주석을 달았다.

3. 추출 결과 (`features.csv`)

파일명	비트수	섹션수	최대엔트로피	고엔트로피섹션명	ImportDLL 수	의심API수	매칭된의심 API
normal_01.exe	32	1	3.7972	.data	0	0	
normal_02.exe	32	1	8.0	.data	0	0	
normal_03.exe	32	3	4.9516	.text	2	0	
packed_fixture.exe	32	1	5.7511	.data	0	0	
packed_upx_like.exe	32	3	5.5786	UPX1	1	2	GetProcAddress;LoadLibraryA
suspicious_api.exe	32	3	5.3294	.text	2	6	CreateRemoteThread;GetProcAddress;LoadLibraryA;RegSetValueExA;VirtualAlloc;WriteProcessMemory

6개 샘플 모두 `MZ` 및 `PE\0\0` 시그니처가 확인되었고, `Machine=0x14C` , `Magic=0x10B` 로 전부 32비트(PE32) 실행 파일이다. 64비트 샘플은 없었다.

3.1 섹션 상세 (보고서 근거용 보조 데이터)

파일명	EntryPoint	섹션	RawSize	Characteristics	권한	엔트로피
normal_01.exe	<code>0x0</code>	.data	8192	<code>0x40000040</code>	R	3.7972

파일명	EntryPoint	섹션	RawSize	Characteristics	권한	엔트로피
normal_02.exe	0x0	.data	8192	0x40000040	R	8.0000
normal_03.exe	0x1010	.text	1024	0x60000020	R+X	4.9516
		.rdata	512	0x40000040	R	2.9032
		.data	512	0xC0000040	R+W	2.7710
packed_fixture.exe	0x0	.data	512	0x40000040	R	5.7511
packed_upx_like.exe	0x9010	UPX0	0	0xC0000080	R+W	0.0000
		UPX1	512	0xE0000020	R+W+X	5.5786
		.rdata	512	0x40000040	R	1.0098
suspicious_api.exe	0x1010	.text	1024	0xE0000020	R+W+X	5.3294
		.rdata	512	0x40000040	R	3.6718
		.data	512	0xC0000040	R+W	0.0000

권한 표기는 `pefile.SECTION_CHARACTERISTICS` 상수로 직접 디코딩한 결과다 (R = `IMAGE_SCN_MEM_READ`, W = `IMAGE_SCN_MEM_WRITE`, X = `IMAGE_SCN_MEM_EXECUTE`).

4. 샘플별 판정과 근거

4.1 normal_01.exe : 정상

- 섹션 1개(`.data` , 8192바이트)뿐이고 `.text` 같은 코드 섹션이 없다. 섹션 권한도 읽기 전용(`0x40000040`)이라 실행 가능한 영역이 아예 없다.
- `AddressOfEntryPoint = 0x0` 으로, 실행 시작 주소가 지정되어 있지 않다.
- Import 디렉터리가 존재하지 않는다(ImportDLL수 0, 의심API 0).
- 엔트로피 3.7972로 낮다. 파일 오프셋 `0x200` 부터 문자열 `"Day 1 benign text fixture."` 가 **300회 반복**되어 있는데, 평문 영문 텍스트가 반복되면 이 정도 엔트로피가 나온다.
- **판정: 정상.** 실행 코드도, 외부 API 호출 능력도, 악성 지표 문자열도 없는 단순 텍스트 데이터 보관용 스텝 파일이다.

4.2 normal_02.exe : 정상 (엔트로피 최댓값 오탐 사례)

- 구조는 `normal_01.exe` 와 동일하다. 섹션 1개(`.data` , 8192바이트, 읽기 전용), `AddressOfEntryPoint = 0x0` , Import 없음.
- **최대엔트로피가 8.0으로 이론적 상한값이다.** 통상 이 수치는 암호화되었거나 압축(패킹)된 데이터의 특징으로 해석된다.

- 그러나 `.data` 섹션의 실제 바이트를 확인한 결과, 이 섹션은 `bytes(range(256))` 즉 `00 01 02 03 ... FE FF` 가 정확히 32회 반복된 순차 램프(ramp) 테이블이었다 (파이썬으로 `섹션데이터 == bytes(range(256))*32` 를 평가해 `True` 확인). 256개 바이트 값이 각각 정확히 32회씩 등장하므로 확률이 완전히 균등해져 엔트로피가 최댓값에 도달한 것이지, 무작위 데이터이기 때문이 아니다.
- **판정: 정상.** 엔트로피 단독 지표의 한계를 보여주는 사례이며, 상세 논의는 6장에서 다룬다.

4.3 normal_03.exe : 정상

- 3개 샘플의 정상군 중 유일하게 실제로 동작하는 프로그램 구조다. 섹션 3개 (`.text` / `.rdata` / `.data`)를 갖추고 `AddressOfEntryPoint = 0x1010` 이 `.text` 섹션 범위(VA `0x1000` , VirtualSize `0x280`) 안을 가리킨다.
- `.text` 권한이 `0x60000020` (읽기+실행, 쓰기 불가)으로, 정상적인 컴파일러가 생성하는 코드 섹션의 표준 형태다.
- Import: `KERNEL32.dll` (`CreateFileW`, `ReadFile`, `WriteFile`, `CloseHandle`, `GetLastError`), `msvcrt.dll` (`printf`, `malloc`, `free`). 총 8개 API 전부 파일 입출력과 표준 C 런타임 기능이며 의심 API 목록과 겹치는 것이 하나도 없다 (의심API 0).
- 문자열도 `"Configuration loaded."` (`0x706`), `"Report written."` (`0x71C`), `"C:\ProgramData\training\report.log"` (`0x72C`), `"training build 1.0.4"` (`0x800`) 로, 로그 파일 하나를 기록하는 프로그램의 것으로 보인다. 네트워크 주소, 레지스트리 경로, 명령어 문자열은 없다.
- **판정: 정상.**

4.4 packed_fixture.exe : 악성 의심

- 섹션이 `.data` 1개(512바이트)뿐이고 Import 디렉터리가 완전히 비어 있다 (Import RVA = 0, Size = 0). 정상적으로 동작하는 실행 파일이라면 최소한 `KERNEL32.dll` 정도는 Import해야 하는데 그것조차 없다. 과제 팀에서 지정한 "패킹된 샘플은 Import 테이블이 거의 비어 있으며, 비어 있다는 사실 자체가 특징"에 해당한다.
- 문자열에도 정상적인 DLL/API 이름이 전혀 없다(`KERNEL32` , `LoadLibrary` 등을 직접 검색했으나 미존재). 대신 `c`dbfaec` (`0x202`), `WPTRVQUS` (`0x222`), `0HLJNIMK` (`0x262`) 처럼 문자 치환/시프트로 난독화된 것으로 보이는 조각만 남아 있다.
- 엔트로피는 5.7511로 "명백한 패킹"이라고 하기에는 애매한 값이다(통상 패킹 판정 임계값은 7.0 이상을 쓴다). 즉 이 샘플은 엔트로피만으로는 잡히지 않고, Import 공백과 문자열 난독화라는 별도 지표로 잡힌다.
- **판정: 악성 의심.** 단, 정직하게 덧붙이면 `AddressOfEntryPoint = 0x0` 이고 실행 가능한 코드 섹션이 없으므로 이 파일은 현 상태로는 어떤 코드도 실행할 수 없다. 실제 악성 행위를 수행하는 완성된 악성코드라기보다는, 패킹/난독화 탐지 실습을 위해 만들어진 불완전한 PE로 보는 것이 더 정확하다. 다만 정적 스크리닝 관점에서 "Import 완전 공백 + 난독화 문자열"은 명백한 이상 신호이므로 정상으로 분류할 수는 없다.

4.5 packed_upx_like.exe : 악성 의심

- `file` 명령이 이 파일을 "UPX compressed"로 인식했고, 실제 섹션 이름이 `UPX0` , `UPX1` , `.rdata` 로 UPX 패커의 전형적 구조와 일치한다. 파일 내에 UPX 패커의 식별 문자열 `UPX!` 도 오프셋 `0x470` 에 존재한다.
- `UPX0` 섹션은 RawSize가 0인데 VirtualSize는 `0x8000` (32KB) 이다. 디스크에는 실체가 없고 메모리에만 큰 공간을 확보하는 구조로, 실행 시점에 압축이 풀릴 영역임을 뜻한다.
- `AddressOfEntryPoint = 0x9010` 은 `UPX1` 섹션 범위(VA `0x9000` , VirtualSize `0x14E`) 안에 있다. 즉 실행은 압축 해제 스텝에서 시작된다.

- Import가 `KERNEL32.dll` 의 `LoadLibraryA` , `GetProcAddress` **단 2개뿐**이며 둘 다 의심 API 목록에 포함된다. 이 조합은 압축을 푼 뒤 실제 필요한 API 주소를 런타임에 동적으로 찾아 쓰기 위한 것으로, 원본 Import 목록을 정적 분석에서 감추는 전형적 수법이다.
- UPX1 섹션 권한이 `0xE0000020` , 즉 **읽기+쓰기+실행이 모두 가능(W+X)** 하다. 정상 파일인 `normal_03.exe` 의 `.text` 가 `0x60000020` (쓰기 불가)인 것과 뚜렷이 대비되며, 자기 자신을 메모리에서 고쳐 쓰는 언패킹 스텝의 특징이다.
- **판정: 악성 의심.** 다만 UPX 자체는 정상 소프트웨어도 사용하는 합법적 압축 도구이므로, 패킹 사실만으로 악성을 단정할 수는 없다. 원본 API 목록이 가려져 있어 정적 분석으로는 실제 기능을 확인할 수 없다는 점이 의심 근거이며, 확정을 위해서는 언패킹 후 재분석이 필요하다.

4.6 suspicious_api.exe : 악성 의심 (가장 강한 정황)

- 6개 샘플 중 의심API수가 6개로 가장 많다: `VirtualAlloc` , `WriteProcessMemory` , `CreateRemoteThread` , `LoadLibraryA` , `GetProcAddress` , `RegSetValueExA` .
- **프로세스 인젝션 정황:** `OpenProcess` (대상 프로세스 핸들 획득, 의심 목록에는 없으나 인젝션에 필수) + `VirtualAlloc` (메모리 확보) + `WriteProcessMemory` (대상 메모리에 코드 기록) + `CreateRemoteThread` (대상 프로세스에서 실행)가 모두 Import되어 있다. 이는 프로세스 인젝션의 교과서적 API 조합이다.
- **지속성 정황:** `ADVAPI32.dll` 의 `RegCreateKeyExA` , `RegSetValueExA` 와 함께 문자열 `"Software\Microsoft\Windows\CurrentVersion\Run"` 이 오프셋 `0x758` 에 존재한다. 이 레지스트리 경로는 로그인 시 자동 실행 등록에 쓰이는 대표적 지속성 확보 위치다.
- **네트워크 통신 암시:** 문자열 `"http://training.invalid/checkin"` 이 오프셋 `0x738` 에 있다. 다만 `InternetOpenA` 를 포함해 네트워크 관련 API는 Import에 전혀 없음을 직접 확인했으므로(문자열 검색에서도 미존재), 이 URL은 실제로 호출되는 기능이 아니라 문자열로만 존재한다. `.invalid` 는 실제로 존재할 수 없도록 예약된 TLD이므로 교육용으로 안전하게 만들어진 것으로 보인다.
- **섹션 권한:** `.text` 가 `0xE0000020` 으로 **쓰기+실행이 동시에 가능(W+X)** 하다. 정상 샘플 `normal_03.exe` 의 `.text` (`0x60000020` , 쓰기 불가)와 대비되는 이상 징후다.
- 엔트로피는 최대 5.3294(`.text`)로 특별히 높지 않다. **엔트로피 임계값만 보는 탐지 규칙이었다면 이 샘플은 걸러지지 않았을 것이며**, Import/API 분석이 왜 반드시 병행되어야 하는지 보여주는 사례다.
- **판정: 악성 의심.** 프로세스 인젝션 API 조합, 레지스트리 Run 키 지속성, 외부 통신 문자열, W+X 섹션 권한이 동시에 관찰되므로 6개 중 가장 확신도가 높다. 다만 Import 목록의 존재는 "그런 기능을 호출할 수 있다"는 것이지 "실제로 그 행위를 수행했다"는 증거가 아니므로, 확정 판정에는 동적 분석이 필요하다.

4.7 보조 지표: 섹션 권한 플래그 비교

추출 과정에서 엔트로피/Import 외에 유용한 제3의 지표를 확인했다. 정상 실행 파일과 의심 파일의 코드 섹션 권한이 뚜렷이 갈린다.

파일	코드 섹션	Characteristics	쓰기 가능	실행 가능
normal_03.exe (정상)	<code>.text</code>	<code>0x60000020</code>	불가	가능
packed_upx_like.exe (의심)	UPX1	<code>0xE0000020</code>	가능	가능
suspicious_api.exe (의심)	<code>.text</code>	<code>0xE0000020</code>	가능	가능

일반적인 컴파일러는 코드 섹션을 읽기+실행(R+X)으로만 만든다. 쓰기까지 허용된 W+X 코드 섹션은 실행 중 자기 코드를 수정하겠다는 뜻이므로 언패킹 스텝이나 자기수정 코드에서 주로 나타난다. 이번 6개 샘플에서 이 지표는 정상군과 의심군을 정확히 갈랐다.

5. 생성형AI 검증

과제 요구사항에 따라 생성형AI에게 각 샘플의 악성 여부 판정과 근거를 요청하고, AI가 언급한 API와 섹션이 실제 추출 결과에 존재하는지 대조했다. AI에게 질의할 때는 과제 팁대로 "근거가 된 API 이름과 섹션 이름을 함께 제시하라"고 요구했다. 정상 샘플 3개는 Claude에, 악성 의심 샘플 3개는 ChatGPT에 질의했다.

5.1 Claude 질의 결과 대조 (정상 샘플 3개)

샘플명	AI가 주장한 근거	실제 추출 결과	일치 여부
normal_01.exe	<code>.data</code> 섹션 1개만 존재하고 <code>.text</code> (코드) 없음, Import 테이블 없음, <code>EntryPoint = 0x0</code> , 엔트로피 3.80(낮음), 문자열 "Day 1 benign text fixture." 반복. 실행 코드가 없어 아무 동작도 하지 않는 테스트용 더미 파일로 판단	<code>EntryPoint = 0x0</code> 확인. <code>.data</code> 1개뿐이고 Import 디렉터리 없음 확인. 엔트로피 3.7972(반올림 3.80) 일치. 해당 문자열 300회 반복 확인	일치
normal_02.exe	<code>.data</code> 섹션만 존재, Import 없음, <code>EntryPoint = 0x0</code> , 엔트로피 8.00(최대치). 문자열이 출력 가능 ASCII 문자표의 반복이라 실제 암호화가 아닌 "높은 엔트로피 테스트용 인위적 데이터"로 판단. 엔트로피만 보면 패킹으로 오탐하기 쉬운 케이스라고 단서를 덧붙임	<code>EntryPoint = 0x0</code> , 엔트로피 8.0 확인. 실제 바이트는 <code>bytes(range(256))</code> 이 32회 반복된 순차 램프이며, <code>strings</code> 로는 그중 출력 가능 구간만 보이므로 "ASCII 문자표 반복"이라는 설명과 부합. 구조적 데이터라는 해석도 정확	일치
normal_03.exe	<code>.text</code> / <code>.rdata</code> / <code>.data</code> 3개 섹션, <code>EntryPoint</code> 정상 지정, Import는 <code>KERNEL32.dll</code> (CreateFileW, ReadFile, WriteFile, CloseHandle, GetLastError)과 <code>msvcrt.dll</code> (printf, malloc, free), 문자열 "Configuration loaded.", "Report written.", "C:\ProgramData\training\report.log", "training build 1.0.4" 확인, 악성 지표 API 전무로 정상 판단. 덧붙여 " <code>.rdata</code> 섹션 특성 플래그가 쓰기 가능 (<code>0x40000040</code>)으로 설정되어	<code>EntryPoint = 0x1010</code> (<code>.text</code> 범위 내) 확인. Import 8개 정확히 일치. 문자열 4개 모두 각각 <code>0x706</code> , <code>0x71C</code> , <code>0x72C</code> , <code>0x800</code> 오프셋에서 확인. 그러나 <code>.rdata</code> 의 <code>0x40000040</code> 은 쓰기 가능 이 아니다. <code>pefile.SECTION_CHARACTERISTICS</code> 로 디코딩하면 <code>IMAGE_SCN_CNT_INITIALIZED_DATA</code> + <code>IMAGE_SCN_MEM_READ</code> 뿐이고, 쓰기 권한 비트 <code>IMAGE_SCN_MEM_WRITE</code> (<code>0x80000000</code>)는 꺼져 있는 읽기 전용 섹션이다	부분 불일치 (판정과 대부분의 인용은 정확하나, 섹션 권한 해석이 틀림)

샘플명	AI가 주장한 근거	실제 추출 결과	일치 여부
	있으나 컴파일러 설정 차이일 뿐 악성 지표는 아니다"라고 언급		

AI가 잘못 지목한 항목: `normal_03.exe` 의 `.rdata` 섹션 특성 플래그 `0x40000040` 을 "쓰기 가능"이라고 설명한 부분. 실제로는 쓰기 비트가 꺼진 읽기 전용 섹션이다. 주목할 점은 **16진수 값 자체는 정확히 인용했고 최종 판정("악성 지표 아님")도 맞았지만, 그 값이 무엇을 의미하는지를 잘못 해석했다는 것이다.** 값의 존재 여부만 확인했다면 이 오류는 발견되지 않았을 것이고, 플래그 상수로 직접 디코딩해봤기 때문에 잡아낼 수 있었다.

이 오류가 이번에는 결론을 바꾸지 않았지만, 만약 다른 샘플에서 실제로 위험한 조합(4.7절의 W+X 코드 섹션 같은)을 같은 방식으로 잘못 디코딩했다면 판정 자체가 뒤집힐 수 있었다.

5.2 ChatGPT 질의 결과 대조 (악성 의심 샘플 3개)

샘플명	AI가 주장한 근거	실제 추출 결과	일치 여부
packed_fixture.exe	<code>AddressOfEntryPoint = 0x0</code> , <code>ImageBase = 0x400000</code> , 섹션은 <code>.data</code> 1개(<code>VA 0x1000</code> , <code>VirtualSize 0x161</code> , <code>RawSize 0x200</code> , <code>Characteristics 0x40000040</code>) = <code>READ + INITIALIZED_DATA</code>), <code>Import RVA/Size</code> 모두 0. 실행 코드가 없는 비정상 PE이며 악성 판정 근거는 부족하고 분석 실습용 fixture로 보인다고 판단	<code>EntryPoint</code> , <code>ImageBase</code> , <code>VA</code> , <code>VirtualSize</code> , <code>RawSize</code> , <code>Characteristics</code> , <code>Import RVA/Size</code> 전부 일치. <code>0x40000040</code> 의 플래그 해석 (<code>READ + INITIALIZED_DATA</code>) 도 정확	일치
packed_upx_like.exe	섹션 <code>UPX0</code> (<code>VA 0x1000</code> , <code>VirtualSize 0x8000</code> , <code>RawSize 0</code> , <code>Characteristics 0xC0000080</code>) , <code>UPX1</code> , <code>.rdata</code> . 파일 내 <code>UPX!</code> 매직 문자열 존재. <code>EntryPoint = 0x9010</code> 이 <code>UPX1</code> 내부를 가리킴. <code>Import</code> 는 <code>KERNEL32.dll</code> 의 <code>LoadLibraryA</code> , <code>GetProcAddress</code> 뿐. 패킹된 실행 파일이나 악성 증거 자체는 없다고 판단	전 항목 일치. <code>UPX!</code> 문자열은 오프셋 <code>0x470</code> 에서 실제 확인. <code>EntryPoint 0x9010</code> 은 <code>UPX1</code> 의 VA 범위(<code>0x9000</code> 이상 <code>0x914E</code> 미만) 내부가 맞음	일치
suspicious_api.exe	<code>EntryPoint = 0x1010</code> (<code>.text</code> 내부) , <code>.text</code> <code>Characteristics 0xE0000020</code> , <code>.rdata</code> 가 파일 오프셋 <code>0x600</code> 부터 <code>0x800</code> 구간에 위치. 그 안에 <code>VirtualAlloc (0x69C)</code> , <code>WriteProcessMemory (0x6AC)</code> , <code>CreateRemoteThread (0x6C2)</code> ,	문자열 오프셋 10개 전부 바이트 단위로 일치(직접 검색해 <code>0x69C</code> 부터 <code>0x758</code> 까지 확인). <code>.text</code> <code>Characteristics 0xE0000020</code> 도 정확. <code>VirtualAllocEx</code> 는 실제로 파일 내에 존재하지 않음을 확인했으므로 해당 지적도 정확	일치

샘플명	AI가 주장한 근거	실제 추출 결과	일치 여부
	OpenProcess (0x6D8), LoadLibraryA (0x6E6), GetProcAddress (0x6F6), RegSetValueExA (0x716), RegCreateKeyExA (0x728) , URL 문자열(0x738), Run 키 문자열(0x758)이 순서대로 존 재. 프로세스 인젝션 + Run 키 지 속성 + URL 조합으로 악성 의심 도가 매우 높다고 판단. 단 VirtualAllocEx 가 아닌 VirtualAlloc 이 Import된 점을 지적하며 Import만으로 실행 여부를 단정할 수 없다는 단서 를 덧붙임		

AI가 잘못 지목한 항목: 없음. 세 샘플 모두 Entry Point, 섹션 Characteristics, Import 목록, 문자열의 파일 오프셋까지 재검증 결과와 일치했다.

5.3 AI 검증 종합

세션	대상	검증한 주장	발견된 오류
Claude	정상 3개	섹션 구성, EntryPoint, Import, 엔트로피, 문자열 4종, 섹션 플래그	1건 (normal_03.exe 섹션 권한 해석)
ChatGPT	악성 의심 3개	섹션 구성, EntryPoint, ImageBase, Characteristics, Import, 문자열 오프셋 10종	0건

이번 검증에서 얻은 교훈은 **AI 검증이 두 층위로 이루어져야 한다**는 점이다.

- 존재 확인:** AI가 언급한 API 이름, 섹션 이름, 문자열, 수치가 실제로 존재하는가.
- 해석 확인:** 존재하는 값을 AI가 올바르게 해석했는가.

이번에 발견된 유일한 오류는 두 번째 층위에서만 잡히는 종류였다. 0x40000040 이라는 값은 실제 섹션에 그대로 존재했으므로 존재 확인만으로는 통과했을 것이고, 그 값을 플래그 상수로 직접 디코딩해 "쓰기 가능"이 아님을 확인했을 때 비로소 오류가 드러났다. 따라서 AI의 결론은 물론 근거로 제시한 값의 **의미까지** 원본 데이터로 재현해봐야 검증이 완성된다.

6. 오탐 가능성 논의

6.1 정상 파일인데 지표가 높게 나온 경우: normal_02.exe

normal_02.exe 는 정상 파일인데도 **최대엔트로피가 이론적 상한인 8.0** 으로 6개 샘플 중 가장 높았다. 여기에 Import DLL수 0이라는, 패키징된 샘플과 동일한 지표까지 겹친다. 이 두 지표만 놓고 자동 탐지 규칙(예: "엔트로피 7.5 이상이면서 Import가 비어 있으면 패키징 의심")을 적용하면 이 파일은 **확실하게 오탐된다**.

실제 원인은 다음과 같다.

- `.data` 섹션의 내용은 `00 01 02 ... FE FF` 가 32회 반복되는 순차 램프 테이블이다. 256개 바이트 값이 각각 정확히 32회씩 등장하므로 확률 분포가 완전히 균등해지고, 새넨 엔트로피 정의상 $-\sum (1/256) \cdot \log_2(1/256) = 8.0$ 이 된다.
- 새넨 엔트로피는 각 값의 출현 빈도만 측정하고 바이트가 나열된 순서는 전혀 보지 않는다. 그래서 완전히 규칙적인 데이터(정렬된 룩업 테이블, 문자 변환표, 사인파 테이블 등)도 난수나 암호화 데이터와 똑같이 최댓값에 도달할 수 있다. 이것이 엔트로피 지표의 구조적 한계다.
- Import가 없는 이유도 패킹 때문이 아니라, 이 파일이 애초에 코드 없이 데이터만 담은 스텝이기 때문이다. 즉 같은 신호라도 원인이 전혀 다를 수 있다.

보완 방법으로는 (1) 고엔트로피 섹션이 실행 가능 속성을 가진 코드 섹션인지 데이터 섹션인지 함께 보기, (2) 바이트열에 반복 주기나 순차 패턴이 있는지 검사하기(이번 경우 256바이트 주기가 명확했다), (3) 실행 가능한 코드가 존재하는지 `AddressOfEntryPoint` 로 확인하기 등이 있다. 이번 샘플은 세 가지 모두에서 "정상 데이터"라는 답이 나온다.

6.2 반대 방향의 위험: packed_fixture.exe

반대로 `packed_fixture.exe` 는 엔트로피가 5.7511로, 흔히 쓰는 패킹 임계값 (7.0 이상)에 한참 못 미친다. 엔트로피 기준만 적용했다면 이 샘플은 **미탐지 (false negative)** 되었을 것이다. 이 샘플을 잡아낸 것은 엔트로피가 아니라 "Import 디렉터리가 완전히 비어 있다"는 별개 지표였다.

6.3 정리

`normal_02.exe` 는 높은 엔트로피가 오탐을 만들고, `packed_fixture.exe` 는 낮은 엔트로피가 미탐을 만들며, `suspicious_api.exe` 는 엔트로피가 평범(5.33)한데도 Import 조합만으로 가장 위험한 샘플이었다. 세 사례 모두 같은 결론을 가리킨다. 단일 지표로는 안 되고, 엔트로피 / Import 구성 / 섹션 권한 / 문자열을 교차 검증해야 한다.

7. 결론

샘플	최종 판정	핵심 근거
normal_01.exe	정상	코드 섹션 없음, EntryPoint 0, Import 0, 낮은 엔트로피(3.80), 평문 반복 텍스트
normal_02.exe	정상	엔트로피 8.0이지만 실체는 순차 램프 테이블. 코드/Import 없음 (오탐 사례)
normal_03.exe	정상	표준 파일 입출력/CRT API만 사용, 의심API 0, 코드 섹션 권한 R+X 정상
packed_fixture.exe	악성 의심	Import 완전 공백, 난독화 문자열. 단 EntryPoint 0으로 실행 불가능한 불완전 PE
packed_upx_like.exe	악성 의심	UPX 시그니처(UPX0/UPX1/ <code>UPX!</code>), 최소 Import 2개, W+X 섹션
suspicious_api.exe	악성 의심	프로세스 인젝션 4종 API, Run 키 지속성, 외부 URL, W+X 코드 섹션, 의심API 6개(최다)

정적 분석만으로는 "그런 기능을 호출할 수 있다"까지만 입증되고 "실제로 악성 행위를 수행했다"는 증명되지 않는다. 위 판정은 모두 PE 구조, Import 테이블, 섹션 특성, 문자열에 근거한 정적 판정이며, 확정에는 동적 분석이 필요하다.

생성형AI 검증에서는 총 6개 샘플에 대한 두 AI 세션의 주장을 항목별로 대조해 **1건의 오류**를 발견했다. `normal_03.exe` 의 섹션 권한 플래그 `0x40000040` 을 "쓰기 가능"이라고 해석한 부분으로, 값 자체는 정확했으나 의미 해석이 틀린

경우였다. 나머지 주장들은 문자열의 파일 오프셋 단위까지 실제 데이터와 일치했다.

이 결과가 주는 교훈은 "AI가 대체로 정확하니 믿어도 된다"가 아니라, **AI의 정확도와 무관하게 검증 절차 자체가 필요하다는** 것이다. 이번에 발견된 오류는 값의 존재만 확인해서는 잡히지 않고 그 값의 의미까지 원본 데이터로 재현해봐야 드러나는 종류였다. 생성형AI는 정적 분석을 가속하는 보조 도구이며, 최종 판단의 근거는 항상 직접 추출한 원본 데이터여야 한다.

부록: 검증 재현 방법

```
pip install -r requirements.txt
python extract_features.py          # PE_6/ 를 분석해 features.csv 생성
python extract_features.py <샘플_디렉터리> <출력_csv_경로>
```

본 보고서에 인용된 모든 수치(EntryPoint, 섹션 크기/권한, 엔트로피, Import 목록, 문자열 오프셋)는 `pefile` 파싱과 원시 바이트 검색으로 재확인했으며, `features.csv` 는 재실행 시 동일한 결과가 나오는 것을 확인했다.